

Informe
Trabajo Práctico Integrador
UNQ-Shop

Billordo, Priscila

priscilabillordounq@gmail.com

Escobar, Alison

alisonecobar648@gmail.com

Profesores

Butti, Matias

Cano, Diego

Urbietta, Matias

Programación con Objetos II

1. Introducción

En el presente informe se destacan las decisiones de diseño para el desarrollo de un sistema de ventas online. Nuestro objetivo fue modelar una solución que abarque la gestión de su catálogo de productos, proceso de pedidos, cálculo de costos de envío y pago.

Durante el desarrollo del mismo, se aplicaron los conceptos de los patrones de diseño vistos en la materia como Composite, State, entre otros, permitiendo tener un sistema flexible, mantenible y escalable, teniendo en cuenta la reutilización de código y una buena separación de responsabilidades.

2. Análisis y diseño del sistema

2.1. Catálogo de productos

Para representar los elementos del catálogo, se decidió crear una abstracción común para todos los ítems. La decisión surge porque el sistema debe manejar de la misma forma productos individuales y paquetes, permitiendo que un cliente del catálogo pueda consultar información sobre el ítem, sin importarle si es una consulta para uno u otro.

Para implementar esta estructura, se decidió utilizar el patrón de diseño *Composite*, ya que el dominio requiere representar tanto elementos simples como agrupaciones de elementos. En este caso, un **Producto** funciona como un elemento simple del catálogo, mientras que un **Paquete** representa una composición de otros elementos. Esto permite que un paquete pueda contener productos u otros paquetes, formándose una estructura donde consultas como el precio del paquete pueda resolverse de forma recursiva .

Según la definición del patrón *Composite* de *Gamma et. al.*, la clase abstracta **Item** cumple el rol de *Component/Componente*, ya que establece el contrato común que deben respetar todos los elementos. La clase **Producto** representa el rol de *Leaf/Hoja*, debido a que es el elemento final que no contiene otros objetos. Por otra parte, la clase **Paquete** cumple con el rol de *Composite/Contenedor*, porque mantiene una colección de elementos **Item** y delega parte de su comportamiento a ellos. Finalmente la clase encargada de consultar el catálogo cumple el rol de *Client*.

2.2. Ciclo de vida del pedido

“Un pedido modifica su comportamiento a lo largo de su ciclo de vida. Según el contexto, existen operaciones que son válidas desde él.”

Teniendo en cuenta esto, hemos reconocido el patrón *State*.

En este caso, la clase **Pedido** es el *contexto*. **Pedido** colabora con un *estado* tipado como **EstadoPedido**, una clase abstracta de la cuales las clases concretas **Borrador**, **Confirmado**, **En_Preparación**, **Enviado**, **Entregado** y **Cancelado** heredan su protocolo, estas clases representan los diferentes estados de un Pedido, siendo *estados concretos*. Cada uno de ellos implementa lo que le corresponde, lanzando excepciones en caso de que se llame una operación inválida (por ejemplo, que Cancelado reciba el mensaje **cancelar(Pedido)**).

2.3. Método de envío

En este apartado, se decidió utilizar el patrón de diseño *Strategy*, debido a la existencia de diferentes métodos de envío disponibles que a su vez contienen diferentes lógicas para resolver el cálculo del envío.

Se definió una abstracción en común para los métodos de envío, donde cada uno la implementa con su propio algoritmo de cálculo. Permitiendo encapsular las reglas particulares de cada alternativa.

De esta manera, un envío estándar puede calcular su costo considerando factores como el peso del pedido y la distancia hasta la dirección de entrega, mientras que un envío express utiliza otro criterio basado en el valor del pedido. Por otra parte, el retiro en sucursal posee una lógica diferente donde depende de la disponibilidad del producto en la sucursal seleccionada.

Además, esta decisión evita que la lógica del envío quede mezclada con la lógica propia del pedido, manteniendo cada clase con una única responsabilidad. Si en el futuro se agrega un nuevo método de entrega, solo sería necesario crear una nueva estrategia que implemente el contrato definido, sin modificar el código existente.

Según la definición de *Gamma et. al.*, dentro del patrón *Strategy* la interfaz **MetodoDeEnvio** cumple con el rol de *Strategy*, ya que define el contrato común que deben de implementar las distintas variantes. Las clases concretas como **EnvioEstandar**, **EnvioExpress** y **RetiroSucursal** cumplen el rol de *ConcreteStrategy* porque contienen cada una la

implementación específica del algoritmo correspondiente. Por último, la clase que utiliza estos objetos, Pedido, cumple el rol de *Context/Contexto*.

2.4. Método de pago

“El procesamiento de un pago siempre sigue los mismos pasos en el mismo orden: validar los datos, reservar los fondos, ejecutar la transacción y notificar el resultado. Sin embargo, cada medio de pago implementa esos pasos de manera diferente.”

Se reconoce el patrón **Template Method**, dado que para realizar un pago, sin importar el medio de pago, el procesamiento de este sigue siendo el mismo.

La *clase abstracta* que define la interfaz para todos los medios de pago posibles es **MedioDePago**, que posee el *método plantilla* **procesarPago()** junto a los métodos abstractos como los “pasos” para procesar el pago, cuyas *subclases* (**TarjetaDeCredito**, **TransferenciaBancaria**, **BilleteraVirtual**) redefinen según lo requerido. A su vez, cada medio de pago conoce una API, sin embargo, esto forma parte de la implementación de los distintos medios de pago, fuera de lo que es el patrón en sí, por lo que no será un tema a profundizar.

2.5 Notificaciones del pedido

Para modelar el sistema de notificaciones del pedido, se decidió utilizar el patrón de diseño **Observer**. Esta decisión surge debido a que un cambio en el estado de un pedido puede generar diferentes acciones dentro del sistema, como enviar un correo al cliente, generar una factura o actualizar un sistema de fidelización. Pero el pedido no debería conocer la existencia ni la implementación de cada una de estas acciones.

Mediante este patrón, el pedido actúa como un objeto observable que informa a los objetos interesados cuando ocurre un cambio de estado. Cuando se realiza una transición el pedido comunica con la información necesaria a todos los observadores registrados, permitiendo que cada uno ejecute su comportamiento específico de manera independiente.

Es decir, los observadores reciben el pedido y los estados anterior y nuevo porque la información necesaria para reaccionar se encuentra en el contexto del pedido.

Según la definición de *Gamma et al.*, dentro del patrón **Observer** la clase **Pedido** cumple el rol de *Subject* u objeto observable, ya que mantiene la lista de observadores y se encarga de notificar los cambios. La interfaz **Subsistema** cumple el rol de *Observer/Observador*, estableciendo el

contrato común que deben de implementar los interesados en recibir eventos. Las clases concretas como **NotificadorEmail**, **GeneradorFactura** y **Fidelizacion** cumplen el rol de *ConcreteObserver*, porque contienen la lógica específica que ejecutan cuando reciben una notificación.

2.6. Búsqueda en el catálogo

“Los operadores complejos también son criterios de búsqueda. Esto significa que pueden anidarse libremente para construir consultas de cualquier profundidad.”

El patrón **Composite** se ve reflejado en los criterios de búsqueda del catálogo. Como el cliente no debe diferenciar los criterios por separado (para filtrar los ítems solo usa un criterio) se define el *componente* como la interfaz **CriterioDeBusqueda**, quien se va a encargar de desarrollar un protocolo en común entre los distintos criterios. Los criterios **CriterioPorNombre**, **CriterioPorCategoria**, **CriterioPorPrecioMax** y **CriterioPorDisponibilidad** son las *hojas*. Mientras que **OperadorNOT** y la clase abstracta **OperadorBinario** (con **OperadorAND** y **OperadorOR** como clases concretas que heredan su protocolo) son los *contenedores/composite* que poseen hijos y estos pueden ser otros operadores lógicos u hojas.

El diseño de los criterios es escalable y permite agregar otros criterios a futuro. Por ejemplo, si quisiera tener un criterio por color (un atributo dinámico), defino la clase **CriterioPorColor** que implementa **CriterioDeBusqueda**.

2.7 Reportes

“El sistema debe poder generar reportes sobre el estado del negocio. Cada reporte puede exportarse en más de un formato de salida. El núcleo de cada reporte —qué datos recolectar y cómo estructurarlos— debe ser independiente del formato en el que se presente. Los tres formatos requeridos son: texto plano, CSV y HTML.”

Se decidió utilizar el patrón *Visitor* para separar la lógica de recolección de datos de las clases del dominio, evitando modificar e incorporar responsabilidades a las clases existentes.

La implementación se realiza en *dos etapas independientes*.

En una primera etapa, un visitante recorre las **ventas del sistema** para obtener la información necesaria y construir el contenido del reporte. Los roles del patrón son:

El *Elemento/Element* es la clase **Venta**, que implementa el método **accept()** y permite que un reporte la visite. En segundo lugar, el *Visitante/Visitor* es la interfaz **Reporte**, que define las operaciones que pueden realizarse sobre una venta.

Por otro lado, el *Visitante Concreto/Concrete Visitor* es la clase **ReporteProductosMasVendidos**, encargada de recorrer las ventas, agrupando los ítems y calculando el promedio.

Una vez procesada la información, se aplica una segunda utilización del patrón *Visitor* ya que el contenido de un reporte queda desacoplado del formato en el que será presentado.

En este caso, el *Element* es la interfaz **Reporte**, que define el método **accept()**, mientras que los *Concrete Elements* son los diferentes reportes que pueden surgir a lo largo del tiempo, por ahora solo está definido **ReporteProductosMasVendidos**. A su vez, la interfaz **Formato** es el *Visitor*, que define el método **exportar(Reporte)** y **visitar(Reporte)** que los Visitantes Concretos: **FormatoHTML**, **FormatoCSV** y **FormatoTXT** implementarán. Asimismo, el Reporte sabrá aceptar la visita de los diferentes formatos a exportar.

Con este diseño se logra desacoplar el “núcleo” del reporte (tal como fue requerido) de su formato de exportación. Las clases concretas que implementan **Reporte** no tienen código específico para generar HTML, CSV o texto plano; eso es responsabilidad de los exportadores. Así, si en un futuro quisiera exportarse de otra forma, como por ejemplo PDF o XML, solamente se agrega una nueva clase que implemente **Formato**.

3. Decisiones de diseño

3.1. Reembolsos

“Los reembolsos al cliente simplemente se resuelven generando y registrando una Nota de Crédito en el sistema.”

Cada vez que un Pedido tiene que reembolsar al cliente (esto sucede cuando un Pedido en estado: **EnPreparacion** y **Enviado** es cancelado), se genera una nota de crédito.

Sin embargo, no es el Pedido el que almacena los registros de estas notas, sino el “sistema” que gestiona todo. Por eso mismo, cada vez que un Pedido es cancelado, si su estado llega a ser válido para la operación y puede ser **EnPreparacion** o **Enviado**, el mismo estado crea una instancia de **NotaDeCredito** que recibe un monto (si es **EnPreparacion**, el monto es el costo de los ítems únicamente; si es **Enviado**, el monto es el costo total incluyendo costo de ítems y costo de envío).

Si bien no es muy bien visto que suceda algo como:

```
Enviado >> cancelar(Pedido pedido)
```

```
pedido.reembolsar(new NotaDeCredito(pedido.costoDeItems(),  
pedido)
```

Fue necesario hacerlo de esta manera, ya que una nota de crédito no va a saber diferenciar por sí sola si el monto a reembolsar es el costo de todos los ítems o el costo total. Además, incluimos el Pedido como una relación de conocimiento ya que (si en un futuro) se quisiera solicitar los datos de todos los pedidos que fueron reembolsados, es más eficiente consultar los pedidos de cada nota de crédito.

3.2 Implementación del Observer

Durante el diseño de los subsistemas de notificación, se decidió que solo reaccionen ante los cambios de estado del pedido, sin introducir una dependencia adicional con una entidad **Cliente**. Consideramos que agregar una clase **Cliente** que únicamente contenga datos como correo electrónico u otros atributos personales no aportaba comportamiento relevante al dominio planteado.

Por este motivo, la información necesaria para ejecutar una notificación, como dirección de destino, título o mensaje, es definida por cada observador mediante valores predeterminados.

3.3 Representación de paquetes

Durante el diseño del **Paquete** se decidió modelar a este mismo como ofertas comerciales y no como una caja armada. Por ello, el paquete no tiene un stock propio almacenado; en cambio, su disponibilidad se calcula dinámicamente a partir de los ítems que lo componen.

La cantidad disponible de una oferta corresponde a la cantidad máxima de combinaciones posibles entre sus componentes, determinada por el menor stock disponible de los elementos necesarios.

Por otra parte, se decidió que su categoría no sea almacenada como un atributo independiente, sino que sea obtenida a partir de los elementos que lo componen. Para evitar agregar complejidad adicional al modelo, se estableció que la categoría del paquete corresponde a la categoría de su primer ítem.

3.4 *Prototipo de base de datos*

La clase **EcommerceData** representa una especie de *base de datos* de la **UNQShop**. La idea de crear esta clase surgió al momento de registrar las notas de crédito de un pedido que es cancelado, dado que nos planteamos el siguiente escenario junto a uno de los docentes: “¿Qué pasaría si la **UNQShop** quisiera obtener todas las notas de crédito emitidas?”. Esto implicaría que recorra todos los pedidos históricos consultando si su estado es **CANCELADO** y luego solicitarle a ese **Pedido** su respectiva nota de crédito. Esta solución no es para nada escalable y a su vez es demasiado costosa en términos de rendimiento, por lo que la creación de un **EcommerceData** conocido entre la **UNQShop** y **Pedido**, facilita la gestión y registro de estos documentos.

De forma análoga, debido a la necesidad de distinguir ventas entre los pedidos, cuando un pedido es entregado, se registra una venta en el sistema (**EcommerceData**) lo cual simplifica la generación de reportes, por ejemplo, los reportes de los productos más vendidos.

3.5 *Cliente, sucursal, depósito, dirección, etc.*

No vimos la necesidad de modelar las entidades **Cliente**, **Sucursal**, **Depósito**, **Dirección**, **Comprobante** (cualquier tipo de comprobante) en nuestra solución. Implementar estas clases no es un requerimiento del enunciado propuesto ni impide el correcto funcionamiento del flujo del sistema. Nuestro enfoque estuvo puesto en aquellos conceptos del dominio que representan comportamiento relevante de un modelo de ecommerce centrado en las reglas de negocio y no en detalles estructurales innecesarios.

3.6 *Ventas*

Debido a la restricción de *no modificar las clases ya existentes* para la implementación del Visitor, se tuvo que crear la clase **Venta** que represente un **Pedido** que fue entregado, es decir, que llegó a su estado final. Esto nos da muchísima flexibilidad, ya que una **Venta** solo es instanciada en el momento que un **Pedido** pasa al estado **Entregado**.

La decisión tomada nos permite tener un sistema más ordenado, debido a que el mismo debe saber diferenciar sus pedidos históricos (cancelados, entregados, en proceso) con los pedidos realmente realizados (ventas). Asimismo, nos ayuda a no tener tanto acoplamiento a la hora de sacar la estadística de los ítems vendidos, evitando que un **Item** contenga datos irrelevantes como unidades que se vendieron de ese mismo ítem, siendo algo puramente del entorno de “ventas” más que la esencia de un ítem en sí.

4. Refactoring

4.1. *Criterios de búsqueda simple*

Se aplicó un refactoring introduciendo una clase abstracta

CriterioSimple que define la estructura del método

filtrarItems(List<Item>) como un *Template Method*, donde

condicionDeFiltro() es el método o “paso” que redefinen las subclases

CriterioPorCategoria, CriterioPorDisponibilidad,

CriterioPorNombre, CriterioPorPrecioMax.